Universidad Rafael Landívar
Campus San Alberto Hurtado, S.J. de Quetzaltenango
Facultad de Ingeniería
Ingeniería en Informática y Sistemas
Manejo e Implementación de Archivos
Docente: **Ing. Manuel Rojas**



PROYECTO No. 2

Agenda de Contactos con Acceso Aleatorio

Registros de tamaño fijo, seek() y tell(), índice primario, integridad SHA-256 y respaldos ZIP en Python

Presentado por:

Moshé Arenz Peláez Virula – 1556425

github.com/Arenzzzz/agenda-contactos

Quetzaltenango, 9 de noviembre de 2026

Contenido

1. Introducción 1

1.1. Resumen del proyecto 1

1.2. Datos del proyecto 1

2. Estructura del registro 2

3. Acceso aleatorio con seek() y tell() 3

3.1. Leer un registro (acceso directo) 3

3.2. Agregar al final 3

3.3. Modificar en el lugar 4

3.4. Cómo se muestra tell() en la aplicación 4

4. Índice primario 6

4.1. Búsqueda por ID 6

4.2. Actualización del índice 6

4.3. ¿Por qué es más eficiente que una búsqueda secuencial? 6

4.4. Reconstrucción del índice 7

5. Integridad con SHA-256 8

5.1. Prueba: archivo alterado manualmente 8

6. Respaldo comprimido (ZIP) 11

7. Exportación a JSON y reporte PDF 13

7.1. Exportación a JSON 13

7.2. Reporte PDF 13

8. Información del archivo: metadatos y permisos 15

9. Manejo de errores 17

10. Funcionamiento de la aplicación 22

11. Conclusiones 24

12. Referencias 25

1. Introducción

Este documento describe la **Agenda de Contactos con Acceso Aleatorio**, una aplicación de escritorio en Python que guarda contactos en un archivo de registros de tamaño fijo. La aplicación llega a cualquier registro con **seek()** sin leer el archivo completo, busca por ID mediante un índice primario en JSON, protege los datos con un hash SHA-256 y genera respaldos ZIP.

El código se organiza en tres paquetes: **nucleo/** (registros, acceso aleatorio, índice e integridad), **herramientas/** (exportación, respaldo e información del archivo) e **interfaz/** (ventana en PySide6). El núcleo no depende de la interfaz: toda lectura y escritura de archivos pasa por la clase **Agenda**.

1.1. Resumen del proyecto

Registro	Búsqueda por ID	Respaldo ZIP	Pruebas de error
128 bytes	**1 lectura**	**36.5 % menos**	**10 de 10**
5 campos y fin de línea	índice → seek() → read()	1,206 → 766 bytes	sin que la aplicación se caiga

1.2. Datos del proyecto

Dato	Detalle
Universidad	Universidad Rafael Landívar, Campus San Alberto Hurtado, S.J., Quetzaltenango
Facultad	Ingeniería, Ingeniería en Informática y Sistemas
Curso	Manejo e Implementación de Archivos
Catedrático	Ing. Manuel Rojas
Estudiante	Moshé Arenz Peláez Virula
Carné	1556425
Repositorio	github.com/Arenzzzz/agenda-contactos
Tecnologías	Python 3, PySide6 (interfaz), fpdf2 (PDF); módulos estándar hashlib, zipfile, json y os

2. Estructura del registro

Cada contacto ocupa exactamente 128 bytes en **contactos.dat**. Los largos se definieron en bytes, no en caracteres, porque en UTF-8 la ñ y las vocales con tilde ocupan 2 bytes: si se contaran letras, un nombre con tildes alargaría el registro y descuadraría todas las posiciones.

> **Idea clave:** la longitud fija desperdicia espacio en el relleno, pero a cambio la posición de cualquier registro se calcula con una multiplicación. Eso es lo que permite el acceso aleatorio.

Campo	Bytes	Posición	Justificación
id	8	0–7	Enteros de hasta 99,999,999
nombre	40	8–47	Nombre y dos apellidos con tildes
telefono	15	48–62	Formato internacional, por ejemplo +502 5555-1234
correo	40	63–102	Direcciones de correo comunes
ciudad	24	103–126	Nombres de municipios y ciudades
fin (\n)	1	127	Permite ver un contacto por línea al abrir el archivo
Total	**128**		

La estructura se define una sola vez en **nucleo/config.py**, y el tamaño total se calcula en lugar de escribirse a mano:

```
CAMPOS = {
    "id": 8,
    "nombre": 40,
    "telefono": 15,
    "correo": 40,
    "ciudad": 24
}
FIN_REGISTRO = b"\n"
TAM_REGISTRO = sum(CAMPOS.values()) + len(FIN_REGISTRO)   # 128 bytes
```

Para armar el registro, cada campo se codifica en UTF-8 y se rellena con espacios usando **ljust()** sobre los bytes. Si un campo no cabe, se rechaza con un error en lugar de cortarlo, porque cortar a la mitad de una letra de 2 bytes dejaría bytes inválidos:

```
def _campo_a_bytes(nombre: str, valor, largo: int) -> bytes:
    datos = str(valor).strip().encode(ENCODING)    # texto -> bytes
    if len(datos) > largo:                          # medimos en BYTES
        raise RegistroError(
            f"El campo '{nombre}' ocupa {len(datos)} bytes "
            f"y el máximo es {largo}.")
    return datos.ljust(largo, b" ")
```

3. Acceso aleatorio con seek() y tell()

Como todos los registros miden 128 bytes, el registro n (numerado desde 0) siempre empieza en el byte n × 128. Con **seek()** el programa salta directo a esa posición y con **tell()** confirma dónde quedó el puntero, sin leer los registros anteriores.

Registro n	seek(n × 128)	Bytes que ocupa
0	0	0–127
1	128	128–255
2	256	256–383
7	896	896–1023

El archivo se abre en modo binario (**rb** y **r+b**) porque en modo texto **seek()** y **tell()** no garantizan posiciones en bytes exactos con UTF-8. El contenido sigue siendo texto legible.

3.1. Leer un registro (acceso directo)

leer(n) salta al registro, guarda el valor de **tell()** para mostrarlo en la interfaz y lee exactamente 128 bytes:

```
@staticmethod
def posicion(n: int) -> int:
    return n * TAM_REGISTRO

def leer(self, n: int) -> Contacto:
    if n < 0 or n >= self.cantidad_registros():
        raise IndexError(f"El registro {n} no existe.")
    with open(self.ruta, "rb") as f:
        f.seek(self.posicion(n))            # saltar directo al registro
        self.ultima_posicion = f.tell()     # confirmar la posición
        datos = f.read(TAM_REGISTRO)        # leer exactamente 128 bytes
    return desempaquetar(datos)
```

3.2. Agregar al final

seek(0, os.SEEK_END) lleva el puntero al final; **tell()** devuelve el byte donde empezará el nuevo registro, y ese byte dividido entre 128 es su número de registro:

```
def agregar(self, contacto: Contacto) -> int:
    registro = empaquetar(contacto)             # validar y convertir
    with open(self.ruta, "r+b") as f:
        f.seek(0, os.SEEK_END)                  # mover el puntero al final
        self.ultima_posicion = f.tell()         # ¿en qué byte quedamos?
        f.write(registro)
    return self.ultima_posicion // TAM_REGISTRO # byte / 128 = n
```

3.3. Modificar en el lugar

Se abre con **r+b** (leer y escribir sin borrar) y se escriben 128 bytes encima del registro. El resto del archivo no se reescribe ni cambia de tamaño:

```
def sobrescribir(self, n: int, contacto: Contacto) -> None:
    if n < 0 or n >= self.cantidad_registros():
        raise IndexError(f"El registro {n} no existe.")
    registro = empaquetar(contacto)
    with open(self.ruta, "r+b") as f:
        f.seek(self.posicion(n))        # al inicio del registro
        self.ultima_posicion = f.tell()
        f.write(registro)               # escribir encima, 128 bytes
```

Esto solo es posible con tamaño fijo: el registro nuevo mide lo mismo que el anterior, así que nunca pisa al siguiente. Con registros variables habría que reconstruir el archivo con un temporal.

3.4. Cómo se muestra tell() en la aplicación

La barra de estado muestra el cálculo y el valor real devuelto por **tell()**, que siempre coinciden (por ejemplo **seek(2 × 128) → byte 256 tell() = 256**). El mapa del archivo dibuja cada registro como un bloque y marca con una flecha la posición del puntero.

Idea clave: leer el registro n cuesta siempre un salto y una lectura, sin importar si el archivo tiene 8 o 100,000 contactos.

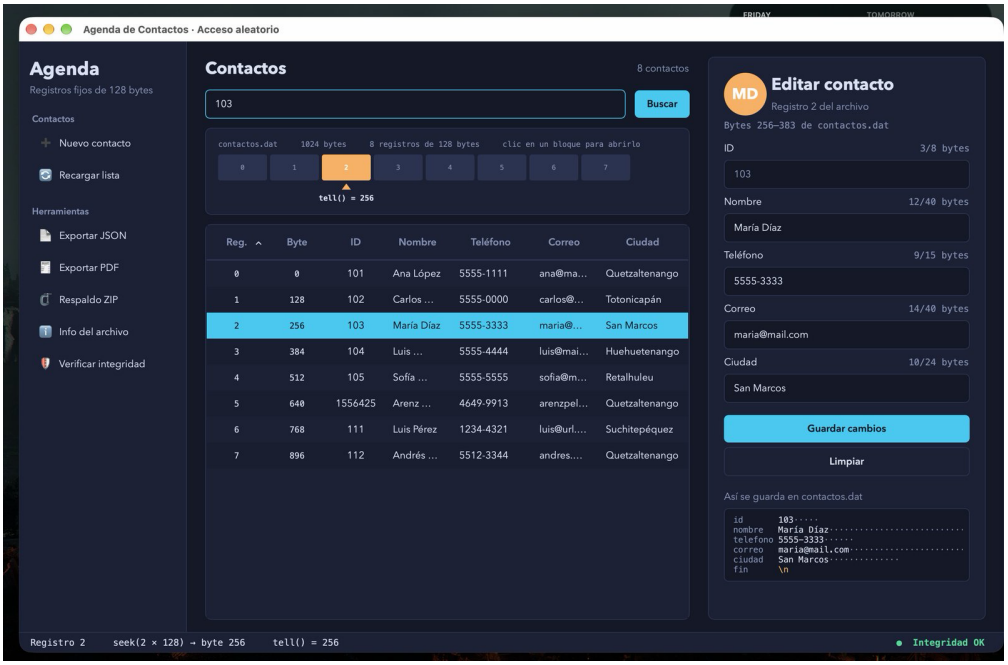


Figura 1. *Búsqueda del ID 103: registro 2, seek(2 × 128) y tell() = 256.*

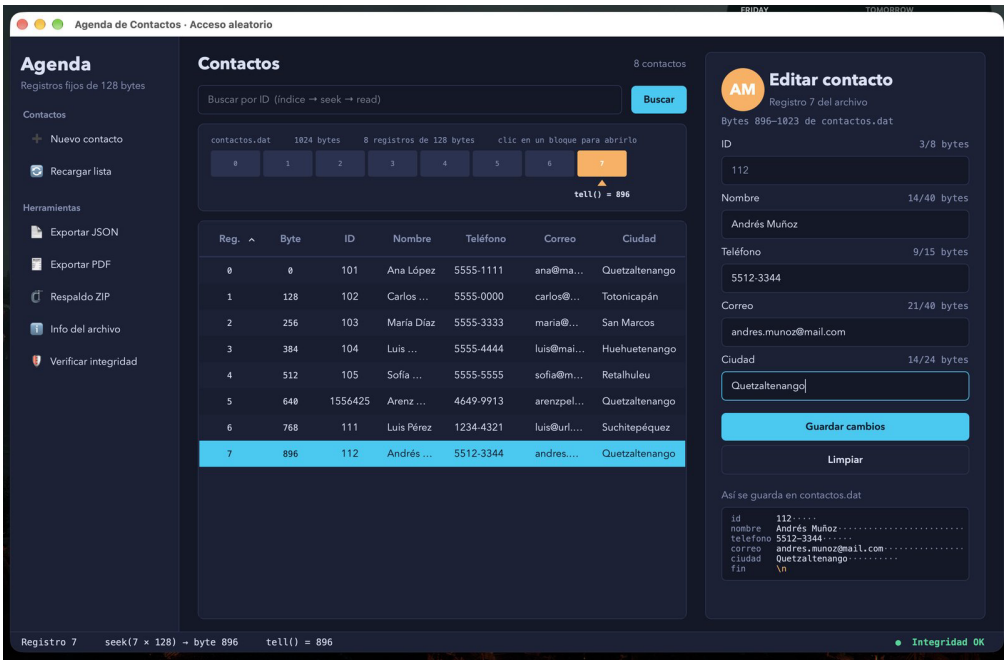


Figura 2. *Contacto agregado al final del archivo con seek(0, SEEK_END).*

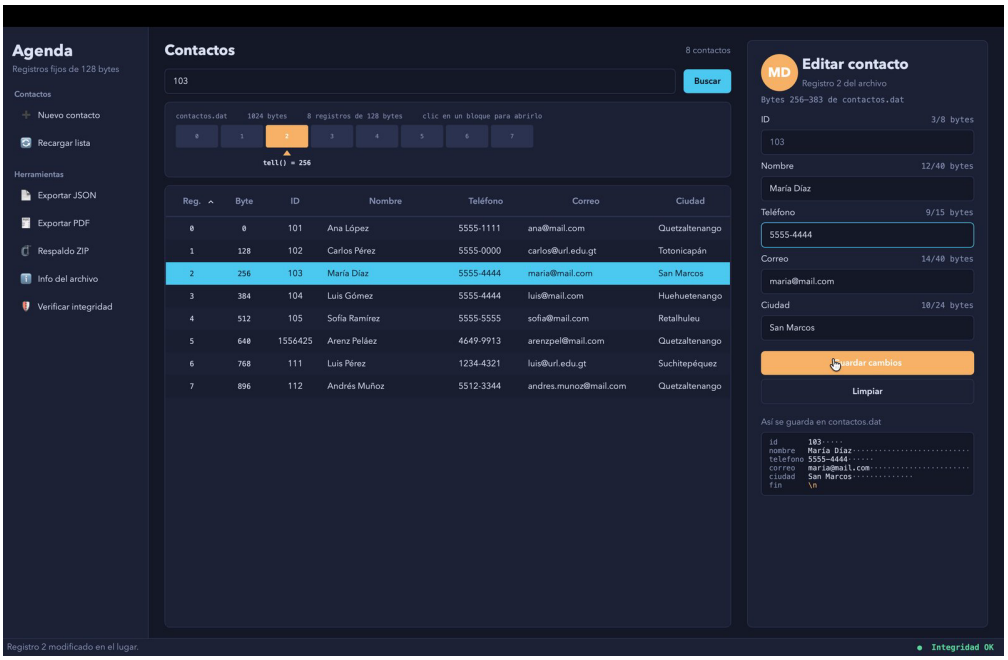


Figura 3. *Modificación en el lugar del registro 2.*

4. Índice primario

El índice es un archivo aparte, **indice.json**, que relaciona cada ID con su número de registro. Es primario porque usa la clave única del contacto: cada ID aparece una sola vez. Este es el índice entregado:

```
{
    "101": 0,   "102": 1,   "103": 2,   "104": 3,
    "105": 4,   "1556425": 5,   "111": 6,   "112": 7
}
```

Al iniciar, la aplicación carga el JSON en un diccionario de Python. Como JSON solo admite claves de texto, al cargarlo cada clave se convierte de nuevo a entero.

4.1. Búsqueda por ID

Primero se consulta el índice y luego se hace **seek()** al registro:

```
def buscar(self, id_contacto: int) -> tuple[int, Contacto]:
    n = self.indice.buscar(id_contacto)     # 1. Consultar el índice
    if n is None:
        raise KeyError(f"No existe un contacto con el ID {id_contacto}")
    contacto = self.archivo.leer(n)         # 2. Acceso directo con seek()
    return n, contacto
```

Por ejemplo, para el ID 103: el índice responde 2, **seek(2 × 128)** lleva el puntero al byte 256 y **read(128)** trae solo ese contacto.

4.2. Actualización del índice

Al agregar, primero se escribe el registro al final de **contactos.dat** y después se registra **id → n** en el índice, que se guarda de inmediato con escritura segura (archivo temporal y reemplazo). Antes de agregar se consulta el índice para rechazar IDs repetidos. El ID no se puede cambiar al modificar, así que modificar nunca desactualiza el índice.

4.3. ¿Por qué es más eficiente que una búsqueda secuencial?

Sin índice, para encontrar un ID hay que leer los registros uno por uno desde el inicio y comparar, porque no se sabe en qué posición está. Con índice, la posición se obtiene de un diccionario y se hace una sola lectura.

	Búsqueda secuencial	Búsqueda con índice
Cómo encuentra el registro	Lee y compara registro por registro	Consulta el diccionario y hace seek()
Lecturas con N contactos (peor caso)	N	1
Con 8 contactos	hasta 8 lecturas (1,024 bytes)	1 lectura (128 bytes)
Con 100,000 contactos	hasta 100,000 lecturas (12.8 MB)	1 lectura (128 bytes)
Si el ID no existe	Recorre todo el archivo para saberlo	Lo sabe sin leer el archivo
Costo adicional	Ninguno	Mantener indice.json al día

La lectura secuencial sigue siendo la adecuada cuando se necesita todo el archivo: llenar la tabla, exportar a JSON y reconstruir el índice.

4.4. Reconstrucción del índice

El índice es un dato derivado: cada registro de **contactos.dat** guarda su propio ID. Si **indice.json** no existe, no es JSON válido o su cantidad de IDs no coincide con la cantidad de registros (índice desactualizado), la aplicación lo reconstruye con una lectura secuencial y avisa al usuario.

```
def reconstruir(self, archivo):
    self.mapa = {}
    for n, contacto in archivo.leer_todos():    # lectura secuencial
        self.mapa[contacto.id] = n              # id -> número de registro
    self.guardar()
```

Idea clave: contactos.dat es la fuente de verdad; el índice es un atajo que siempre se puede volver a construir a partir de él.

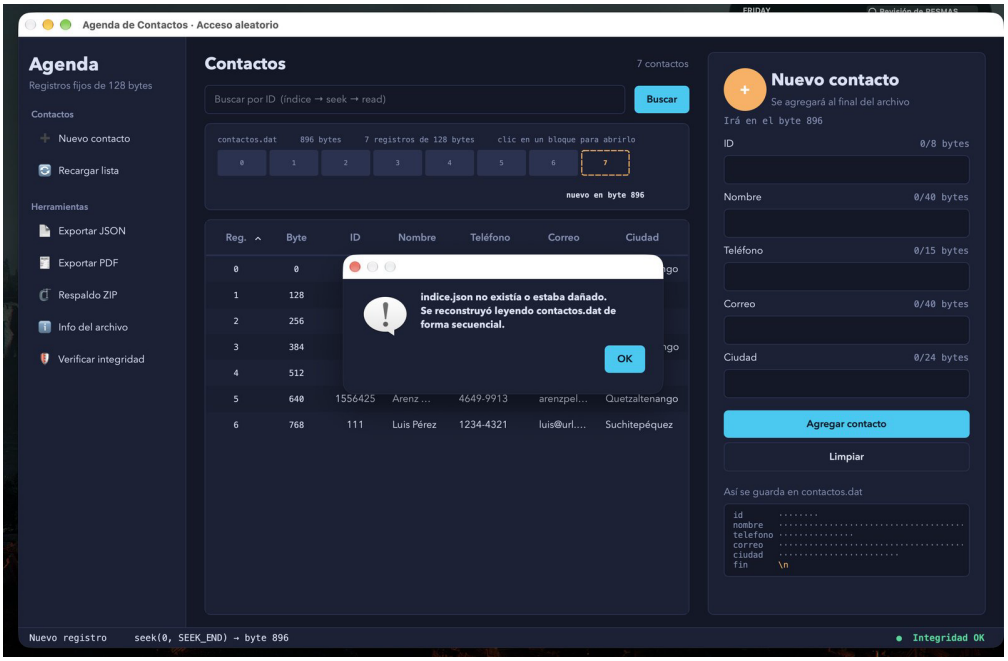


Figura 4. *Índice dañado a propósito: la aplicación lo reconstruye y la integridad sigue en OK.*

5. Integridad con SHA-256

Después de cada cambio hecho desde la aplicación (agregar o modificar) se calcula el hash SHA-256 de **contactos.dat** y se guarda en **contactos.sha256**. Al iniciar, la aplicación lo recalcula y lo compara con el guardado; si no coinciden, avisa que el archivo fue modificado fuera de la aplicación y muestra los dos hashes.

El hash se calcula leyendo el archivo por bloques de 4 KB, para que funcione igual con archivos pequeños o grandes:

```
def calcular_hash(ruta=ARCHIVO_CONTACTOS) -> str:
    sha = hashlib.sha256()
    with open(ruta, "rb") as f:
        while bloque := f.read(4096):   # leer de 4 KB en 4 KB
            sha.update(bloque)          # ir alimentando el hash
    return sha.hexdigest()              # 64 caracteres hexadecimales

def verificar(ruta=ARCHIVO_CONTACTOS, ruta_hash=ARCHIVO_HASH):
    actual = calcular_hash(ruta)
    if not ruta_hash.exists():
        return "sin_hash", "", actual
    with open(ruta_hash, "r", encoding=ENCODING) as f:
        guardado = f.read().strip()
    if guardado == actual:
        return "ok", guardado, actual
    return "alterado", guardado, actual
```

5.1. Prueba: archivo alterado manualmente

Con la aplicación cerrada se cambió "Ana López" por "Ana Lopez " directamente en el archivo:

```
sed -i '' 's/Ana López/Ana Lopez /' datos/contactos.dat
```

La ó ocupa 2 bytes y "o " (o con espacio) también 2, así que el archivo conservó sus 896 bytes y siguió siendo múltiplo de 128. La validación de tamaño no lo habría detectado, pero el SHA-256 sí: un solo byte distinto produjo un hash completamente diferente.

	Hash SHA-256
Guardado por la aplicación	88d700b1814156dd64f9a66c2d4bbac3ed7fd84ab33453403941a3c3b30df462
Calculado tras la alteración	082dc5c2dff4d381cc8f9229854a1fc41bb1408842faf85ccac27e19562bac84

La aplicación mostró el aviso con ambos hashes y preguntó si aceptar el archivo como está. Al responder que no, el indicador de la barra de estado quedó en rojo ("Archivo alterado") y la tabla mostró "Ana Lopez" sin tilde.

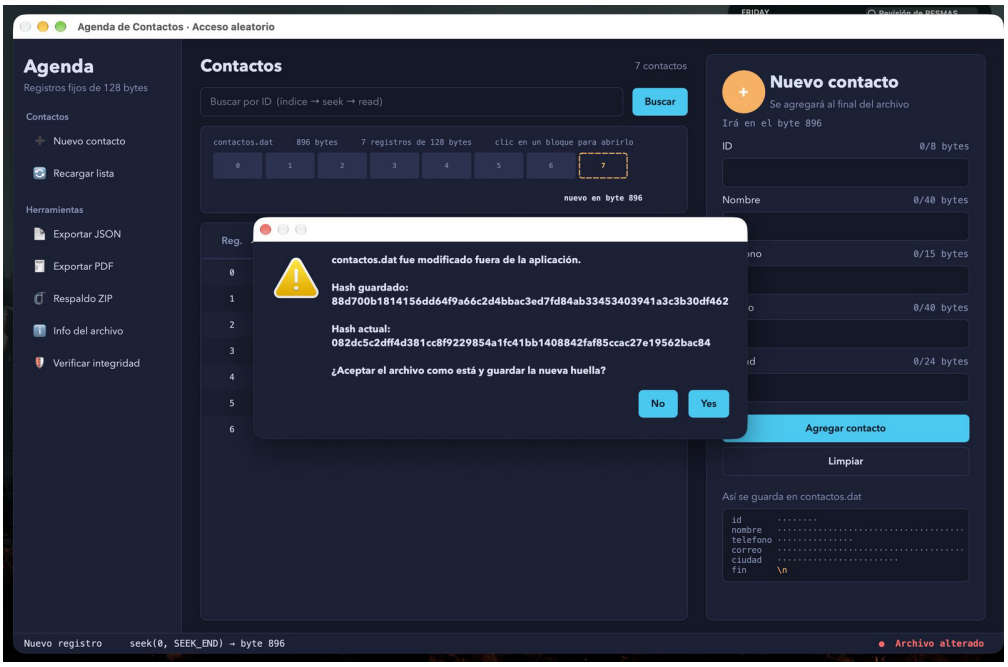


Figura 5. *Aviso de archivo alterado con el hash guardado y el hash actual.*

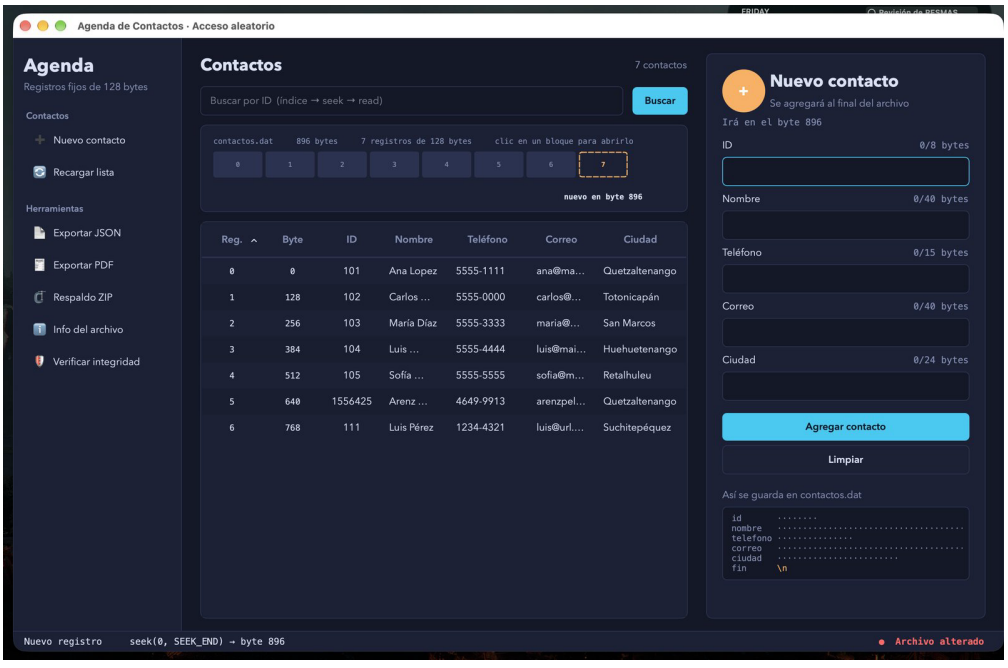


Figura 6. *Ventana principal con el indicador rojo y el nombre alterado.*

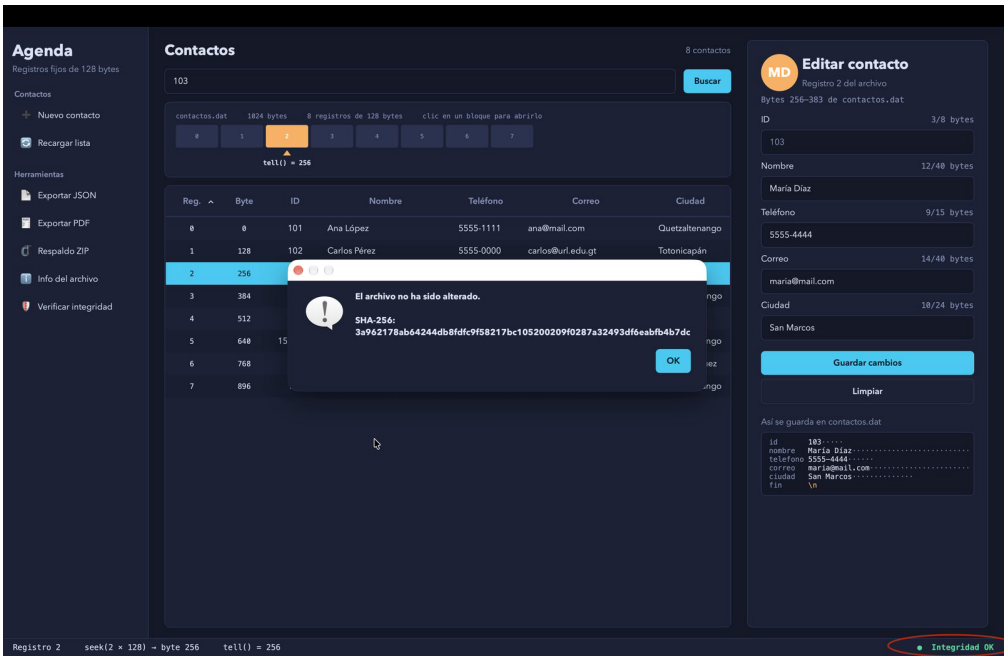


Figura 7. *Verificación de integridad sobre el archivo sin alterar.*

6. Respaldo comprimido (ZIP)

El respaldo entregado, **respaldo_20261106_162324.zip**, redujo 1,206 bytes a 766 bytes (36.5 % menos). Solo **contactos.dat** pasó de 1,024 a 324 bytes (68.4 % menos).

Archivo	Original (bytes)	Comprimido (bytes)	Reducción
contactos.dat	1,024	324	68.4 %
indice.json	118	59	50.0 %
contactos.sha256	64	53	17.2 %
Total / ZIP completo	**1,206**	**766**	**36.5 %**

El ZIP completo (766 bytes) pesa más que la suma de los archivos comprimidos (436 bytes) porque incluye las cabeceras del formato: nombre, fecha y CRC de cada archivo. Con más contactos, ese costo fijo pesa cada vez menos.

contactos.dat se comprime tanto por el relleno de la longitud fija: cada campo termina en espacios repetidos, y el algoritmo DEFLATE guarda esas secuencias como "N veces espacio" en lugar de byte por byte. El hash **contactos.sha256** casi no se comprime porque sus 64 caracteres hexadecimales no se repiten.

El ZIP incluye **contactos.dat** e **indice.json**, como pide el enunciado, y además **contactos.sha256**: al restaurar el respaldo, el hash viene junto con su archivo y la verificación de integridad no da una falsa alarma. Cada respaldo lleva fecha y hora en el nombre para no sobrescribir los anteriores, y al crearlo se vuelve a abrir con **testzip()** para comprobar que se puede leer completo.

```
with zipfile.ZipFile(ruta_zip, "w", compression=zipfile.ZIP_DEFLATED,
                     compresslevel=9) as z:
    for archivo in archivos:
        z.write(archivo, arcname=archivo.name)   # sin carpetas internas

with zipfile.ZipFile(ruta_zip) as z:
    detalle = [(i.filename, i.file_size, i.compress_size) for i in z.infolist()]
    danado = z.testzip()                         # None = todo se lee bien
```

Idea clave: el desperdicio interno de la longitud fija es justo lo que mejor comprime el ZIP.

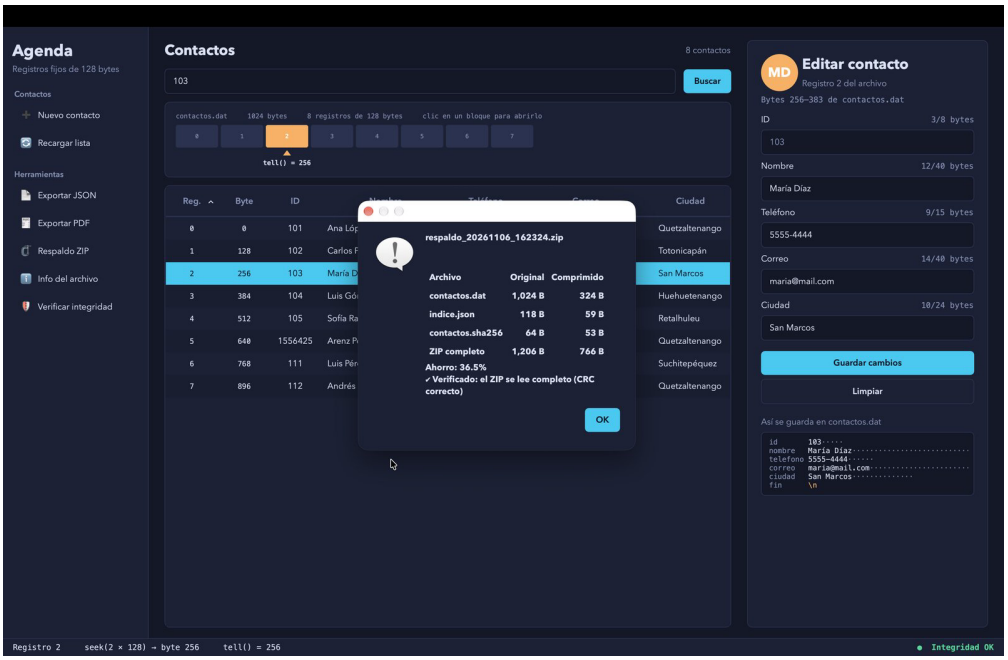


Figura 8. *Respaldo creado: tamaño original contra comprimido y verificación del ZIP.*

7. Exportación a JSON y reporte PDF

La exportación sigue el flujo **contactos.dat → contactos.json → reporte_contactos.pdf**. El JSON entregado se generó el 06/11/2026 a las 16:22:40 con 8 contactos.

7.1. Exportación a JSON

Se leen todos los contactos con lectura secuencial y se guardan junto con metadatos: fecha de generación, archivo de origen, tamaño del registro y total. Se escribe en UTF-8 con **ensure_ascii=False**, para que "López" quede legible, y con escritura segura (archivo temporal y reemplazo).

```
datos = {
    "generado": datetime.now().isoformat(timespec="seconds"),
    "origen": ARCHIVO_CONTACTOS.name,
    "tam_registro": TAM_REGISTRO,
    "total": len(contactos),
    "contactos": [c.a_dict() for c in agenda.listar()],
}
```

7.2. Reporte PDF

El reporte se genera con fpdf2 a partir del JSON, no del **.dat**. Tiene una tabla con ID, nombre, teléfono, correo y ciudad, la fecha de generación tomada del JSON y un pie con el número de página. Desde la aplicación, "Generar reporte PDF" exporta primero el JSON para que el reporte siempre tenga los datos actuales, y luego abre el PDF.

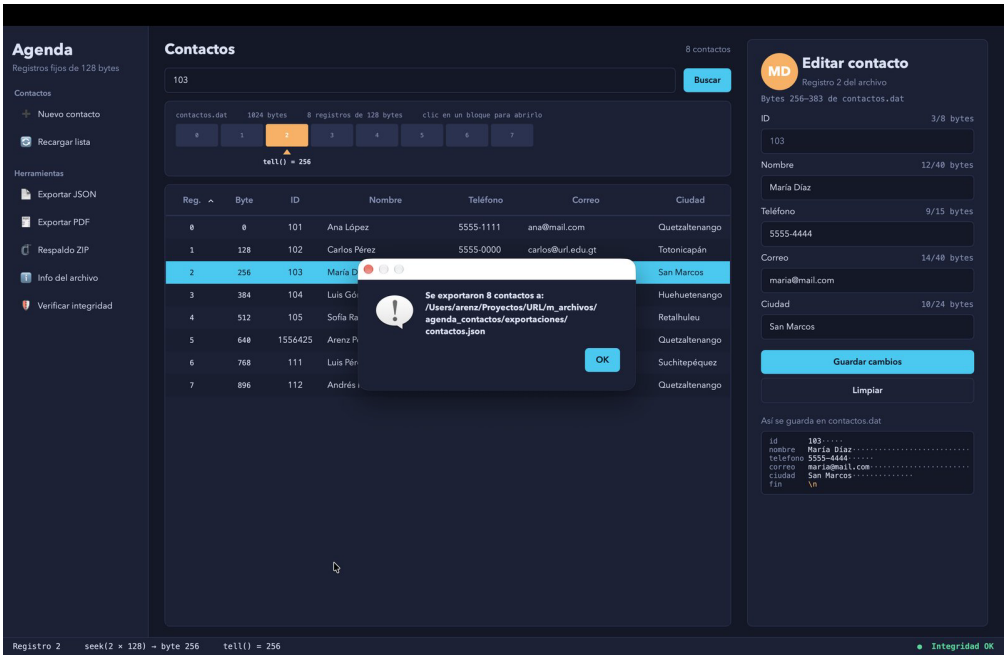


Figura 9. *Aviso de exportación a JSON.*

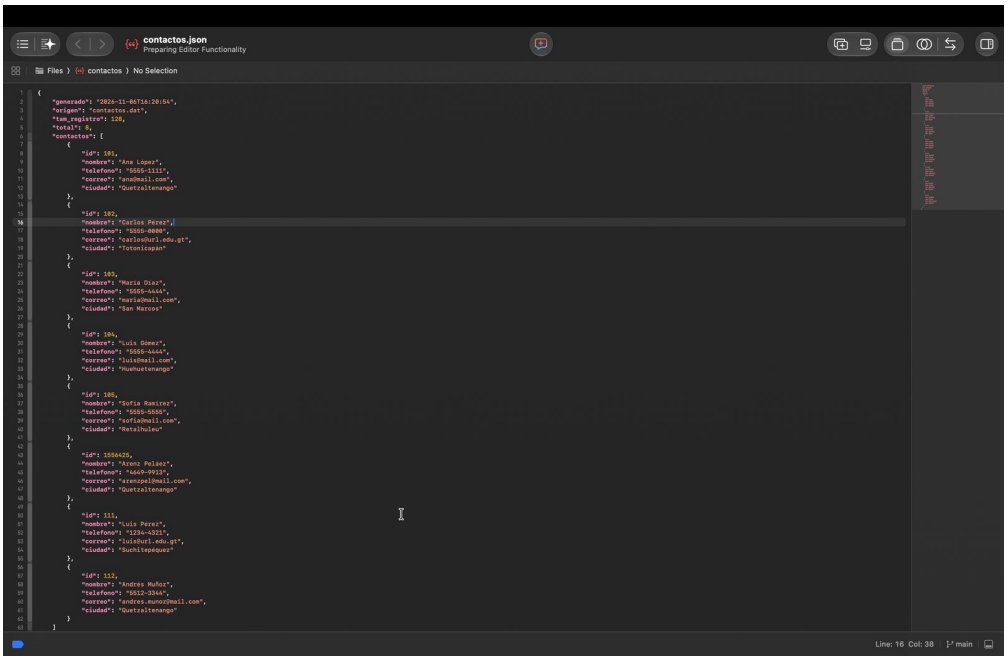


Figura 10. *Contenido de contactos.json con tildes y metadatos.*

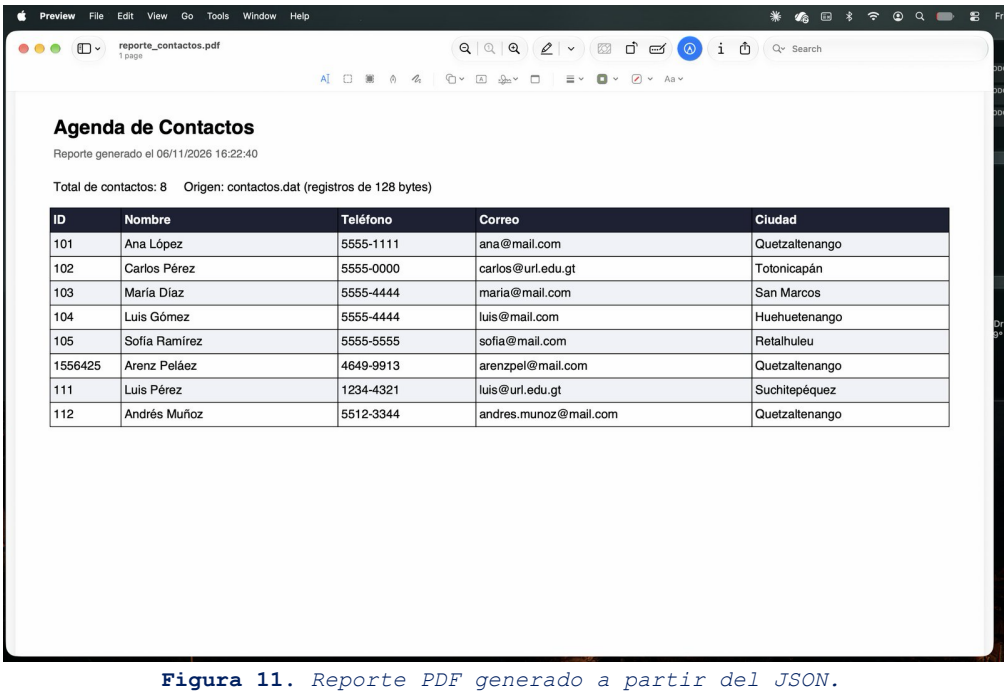


Figura 11. *Reporte PDF generado a partir del JSON.*

8. Información del archivo: metadatos y permisos

La ventana "Información del archivo" muestra lo que el sistema operativo sabe de **contactos.dat** sin leer su contenido. Con el archivo entregado: 1,024 bytes, 8 registros de 128 bytes y 0 bytes sobrantes.

Dato mostrado	De dónde sale
Tamaño en bytes	**os.stat().st_size**
Registros y bytes sobrantes	tamaño // 128 y tamaño % 128 (debe ser 0)
Última modificación y acceso	**st_mtime** y **st_atime**, con **datetime.fromtimestamp()**
Permisos	**stat.filemode(st_mode)** (-rw-r--r--) y en octal (0o644)
¿Se puede leer? ¿Escribir?	**os.access(ruta, os.R_OK)** y **os.access(ruta, os.W_OK)**

os.stat() dice qué permisos tiene el archivo; **os.access()** responde si este programa, con este usuario, puede leerlo o escribirlo en ese momento, que es lo que importa para saber si se podrá guardar.

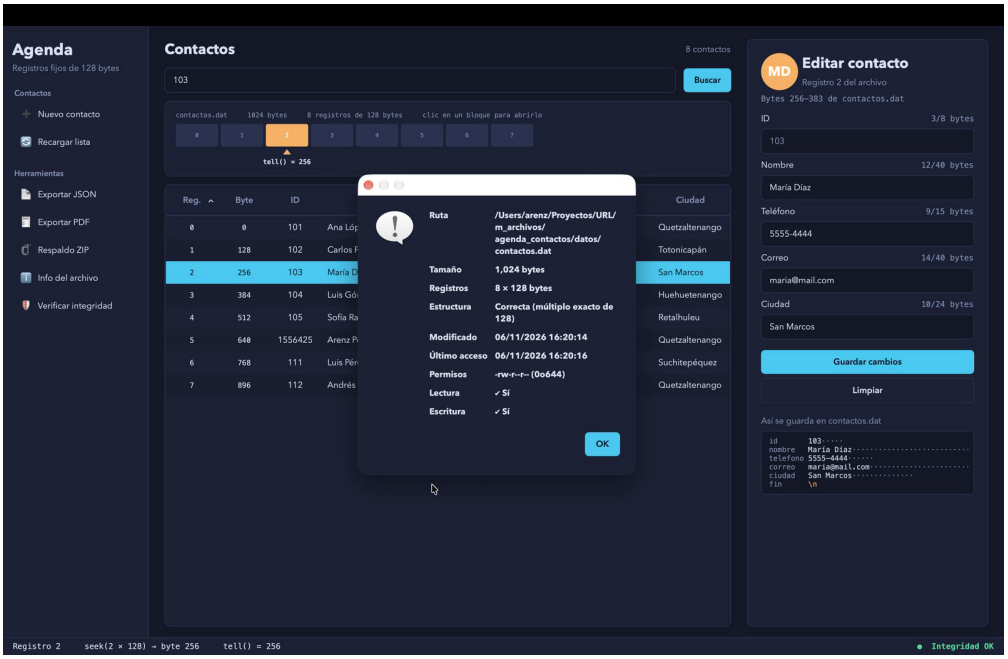


Figura 12. *Información del archivo con permisos normales.*

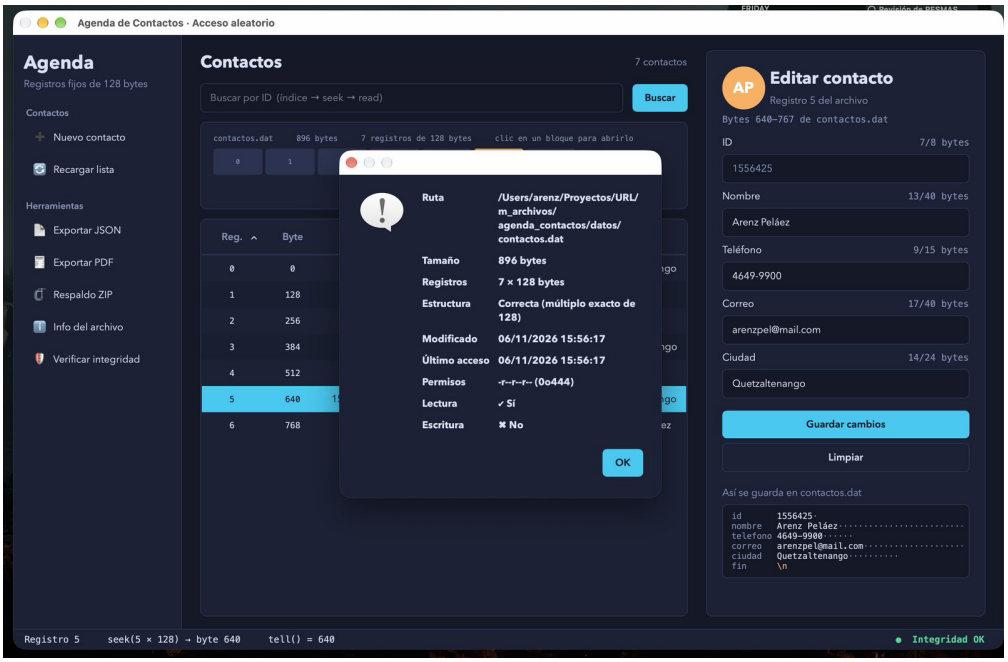


Figura 13. *El mismo archivo en solo lectura (chmod 444): escritura no permitida.*

9. Manejo de errores

La aplicación no se cae en ninguno de los 10 casos probados: en cada uno muestra un mensaje con la causa. Cada caso se preparó con el script **caso.sh**, que primero restaura una copia limpia de los datos y luego provoca el error.

Caso	Cómo se provocó	Resultado	Fig.
Archivo inexistente	Se movió contactos.dat	Aviso; se crea un archivo vacío	14
Archivo corrupto	6 bytes extra al final	"No es múltiplo de 128"; no abre	15
Alterado a mano	"López" → "Lopez ", mismo tamaño	Aviso con los dos hashes	5, 6
Índice dañado	indice.json no es JSON	El índice se reconstruye	4
Solo lectura	chmod 444	Consulta sí; al guardar, "Sin permisos"	16, 13
Sin permisos	chmod 000	Aviso "Sin permisos"; no abre	17
ID inexistente	Buscar el ID 999	"No existe un contacto con el ID 999"	18
ID repetido	Agregar con el ID 101	"Ya existe un contacto con el ID 101"	19
Correo inválido	Correo "luis@"	"El correo no es válido"	20
Campo muy largo	Ciudad de 35 bytes	"ocupa 35 bytes y el máximo es 24"	21

Los errores se detectan en el núcleo y se muestran en la interfaz. **cantidad_registros()** lanza **RegistroError** si el tamaño no es múltiplo de 128; el sistema operativo lanza **PermissionError** al abrir sin permisos; **Agenda.buscar()** lanza **KeyError** si el índice no tiene el ID. Al iniciar, **main.py** atrapa los errores graves antes de crear la ventana:

```
def iniciar_agenda():
    try:
        return Agenda()
    except RegistroError as e:
        QMessageBox.critical(None, "Archivo corrupto", f"...{e}")
    except PermissionError:
        QMessageBox.critical(None, "Sin permisos", "...")
    except OSError as e:
        QMessageBox.critical(None, "Error de archivo", f"...{e}")
    return None
```

PermissionError se atrapa antes que **OSError** porque es un tipo de **OSError**. Todos los archivos de texto (**indice.json**, **contactos.sha256** y el JSON exportado) se leen y escriben con **encoding="utf-8"** explícito.

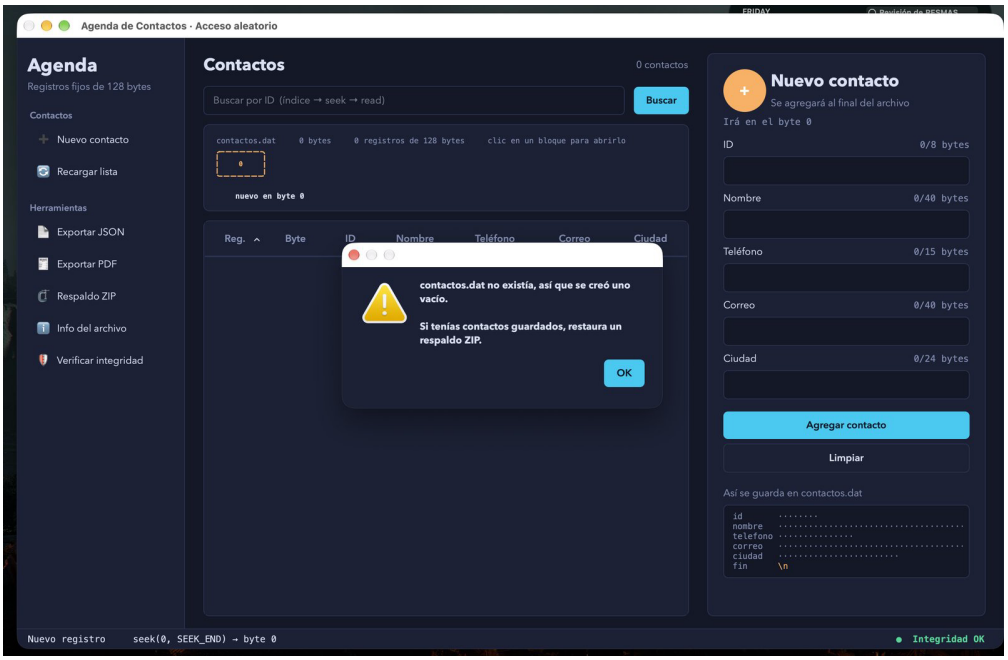


Figura 14. *Archivo de contactos inexistente.*



Figura 15. *Archivo corrupto: tamaño que no es múltiplo de 128.*

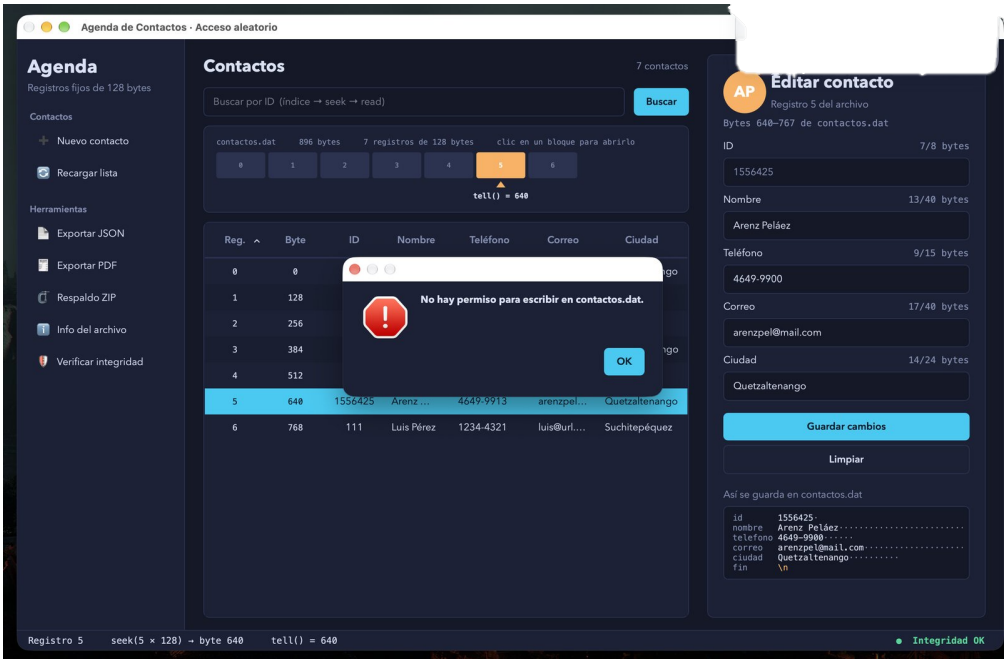


Figura 16. *Intento de guardar sin permiso de escritura.*

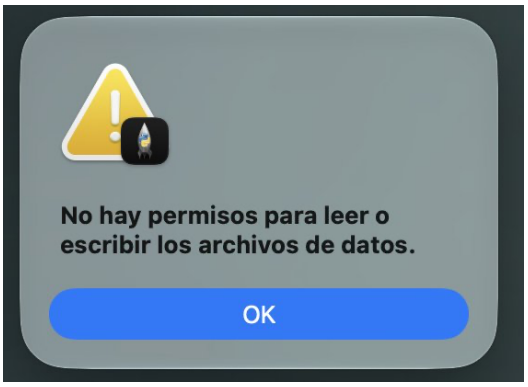


Figura 17. *Archivo sin ningún permiso.*

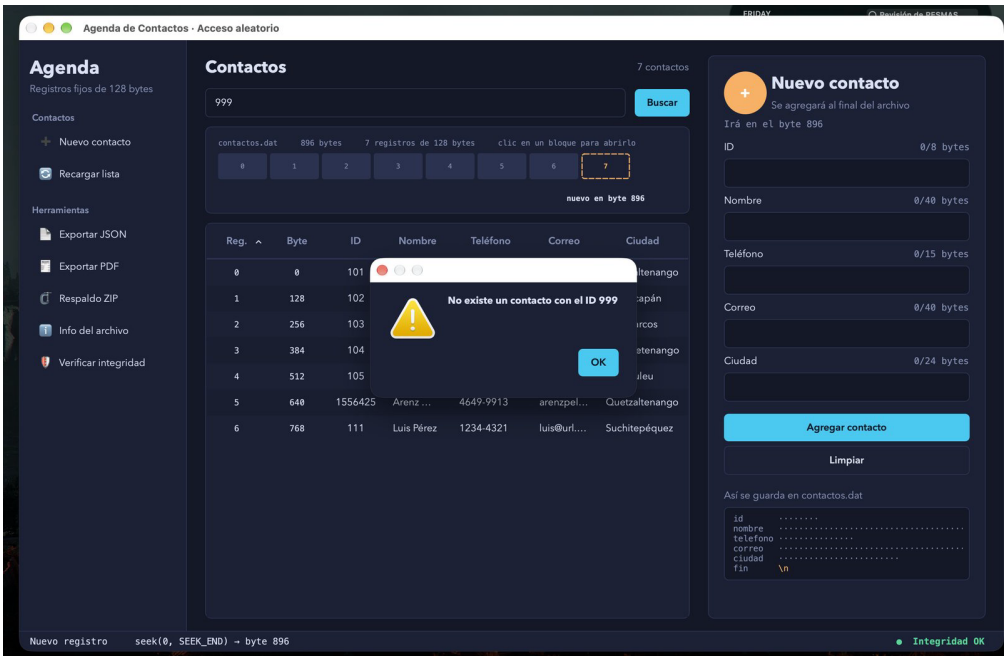


Figura 18. *Búsqueda de un ID inexistente.*

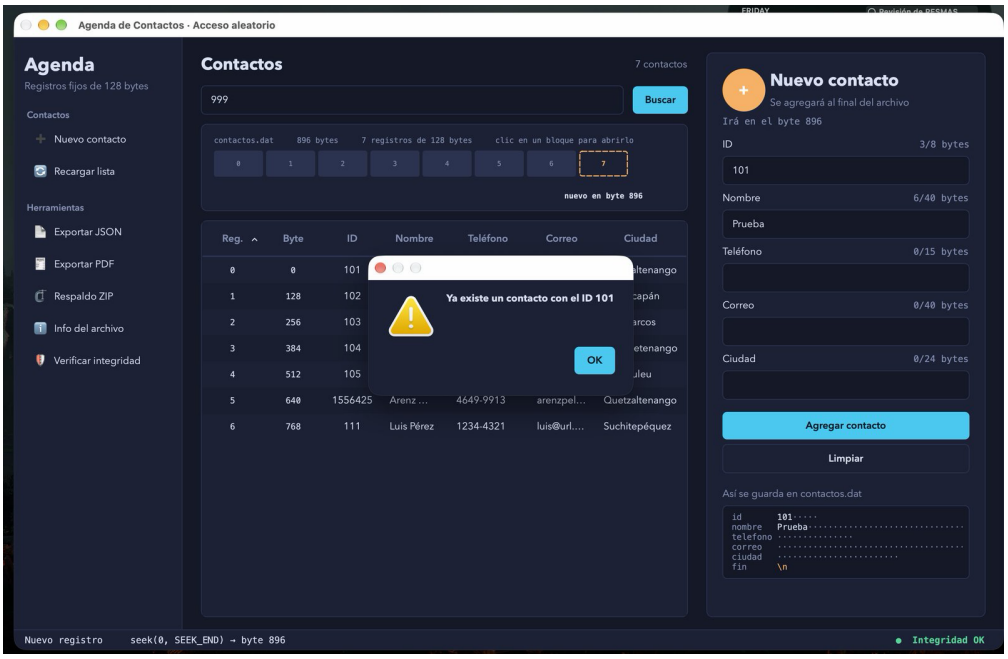


Figura 19. *ID repetido al agregar.*

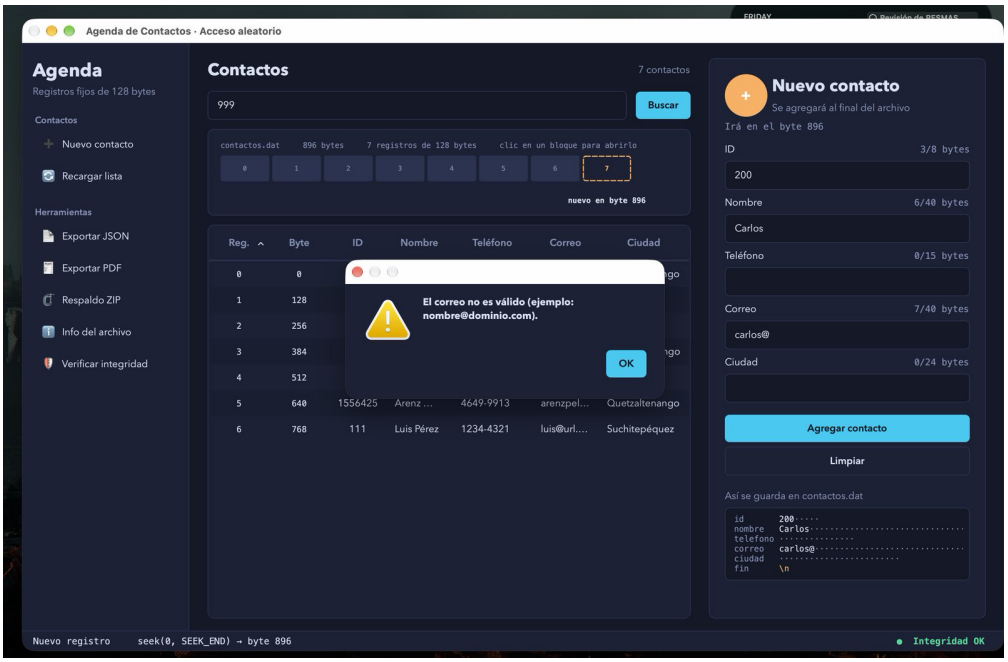


Figura 20. *Correo inválido.*

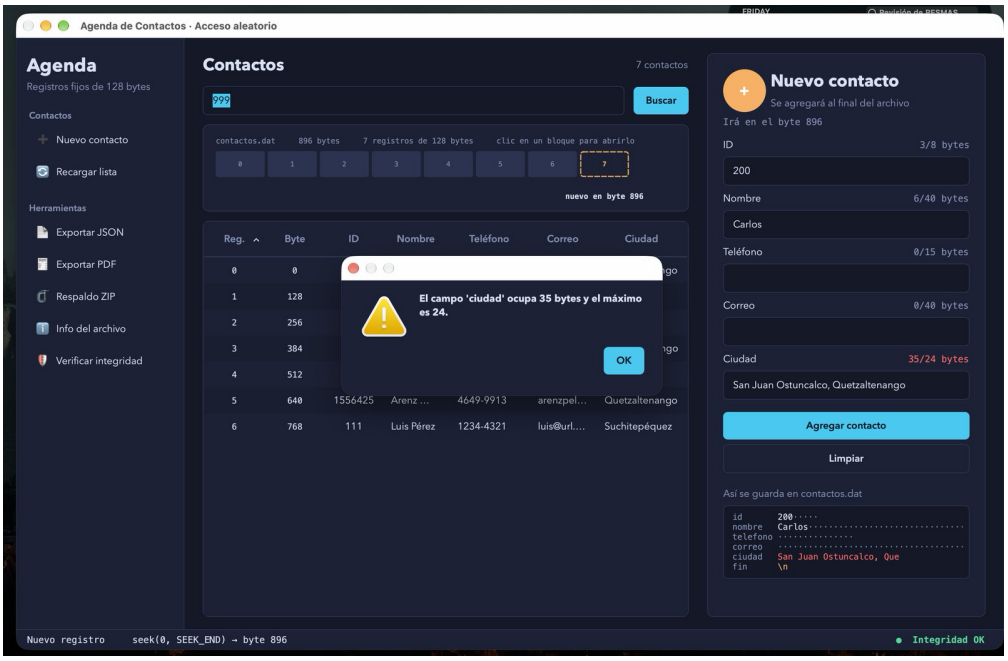


Figura 21. *Campo demasiado largo, con el contador de bytes en rojo.*

10. Funcionamiento de la aplicación

La ventana tiene el menú principal (Contactos, Exportar, Herramientas, Ver y Ayuda), una barra lateral con los mismos accesos, la tabla de contactos con el mapa del archivo y un formulario para agregar o editar. La barra de estado muestra siempre la última posición de **`seek()`** y **`tell()`** y el estado de integridad.

Dos ayudas visuales explican el formato mientras se usa la aplicación:

- **Mapa del archivo:** cada bloque es un registro de 128 bytes. Al pasar el mouse muestra su rango de bytes y su contacto; al hacer clic abre ese registro directamente con **`seek()`**. En modo "Nuevo contacto" marca con línea punteada el lugar donde caerá el registro.
- **Contador de bytes y vista del registro:** cada campo muestra los bytes que usa en UTF-8 ("Andrés Muñoz" tiene 12 letras y ocupa 14 bytes) y se pone en rojo si se pasa del límite. Debajo se ve el registro tal como quedará en el archivo, con el relleno marcado como puntos.

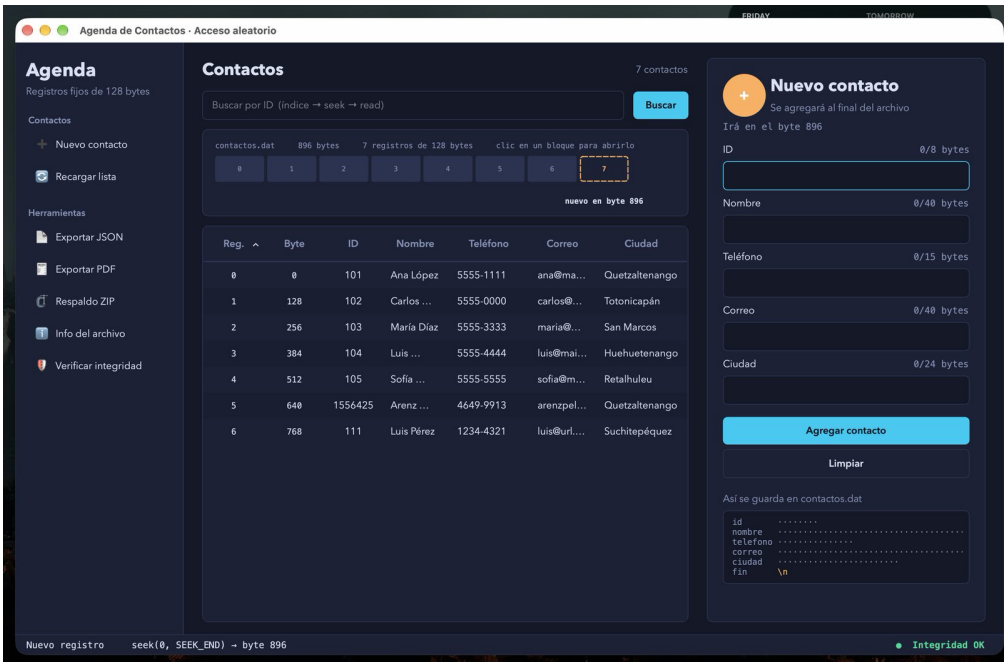


Figura 22. *Pantalla inicial en modo "Nuevo contacto", con el lugar del próximo registro en el mapa.*

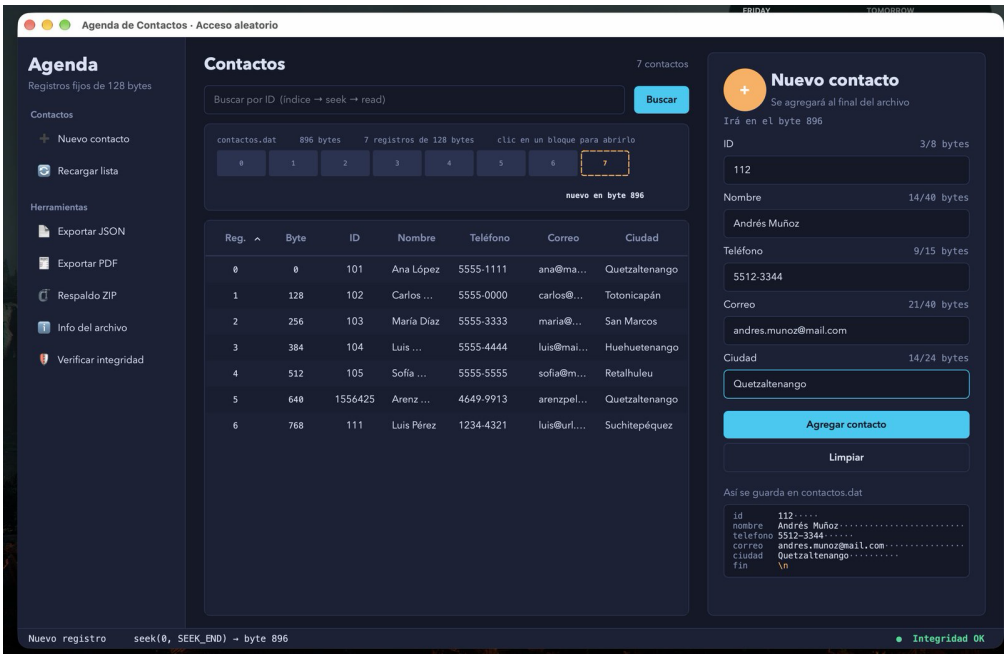


Figura 23. *Formulario con el contador de bytes y la vista del registro.*

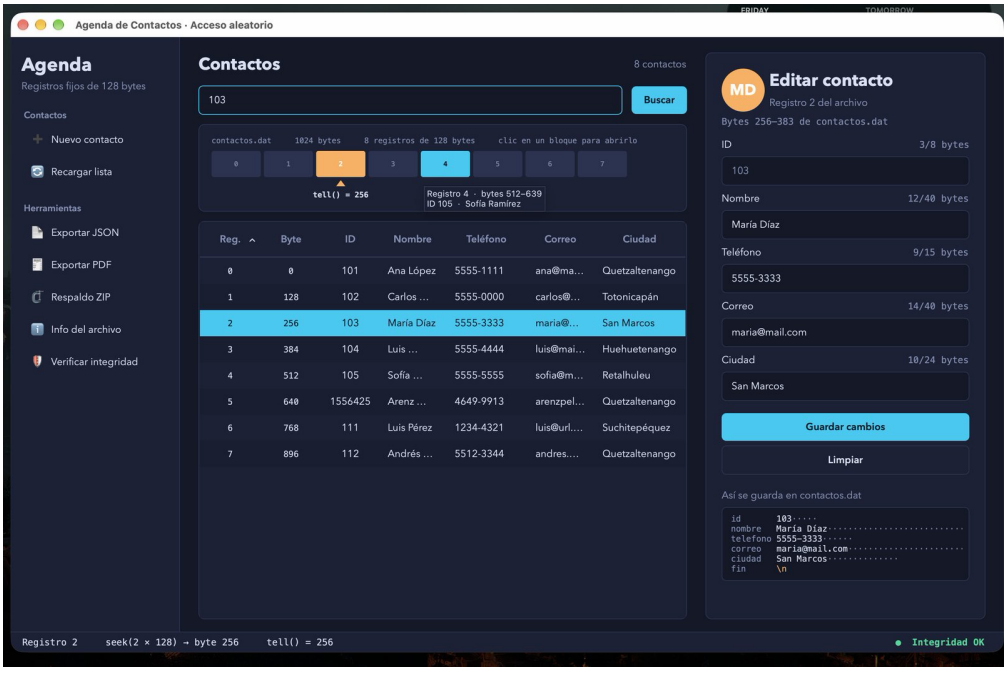


Figura 24. *Mapa del archivo: información del registro al pasar el mouse.*

11. Conclusiones

- Los registros de tamaño fijo permiten calcular la posición de cualquier contacto como n × 128 y llegar a él con un solo **`seek()`**, sin recorrer el archivo.
- El índice primario convierte la búsqueda por ID en una consulta a un diccionario más una lectura de 128 bytes; la búsqueda secuencial necesitaría hasta N lecturas.
- Medir los campos en bytes UTF-8, y no en caracteres, es indispensable para que las tildes y la ñ no descuadren las posiciones.
- El SHA-256 detectó una alteración de un solo carácter que no cambió el tamaño del archivo, algo que la validación de tamaño no podía ver.
- El relleno de la longitud fija, que desperdicia espacio en disco, es lo que permite que **`contactos.dat`** se comprima 68.4 % en el respaldo ZIP.
- Separar el núcleo de la interfaz permitió probar cada parte por su cuenta y manejar los errores en un solo lugar sin que la aplicación se cierre inesperadamente.

12. Referencias

Python Software Foundation. (2026). hashlib — Secure hashes and message digests. https://docs.python.org/3/library/hashlib.html

Python Software Foundation. (2026). io — Core tools for working with streams (seek y tell). https://docs.python.org/3/library/io.html

Python Software Foundation. (2026). json — JSON encoder and decoder. https://docs.python.org/3/library/json.html

Python Software Foundation. (2026). os — Miscellaneous operating system interfaces. https://docs.python.org/3/library/os.html

Python Software Foundation. (2026). zipfile — Work with ZIP archives. https://docs.python.org/3/library/zipfile.html

The Qt Company. (2026). Qt for Python (PySide6). https://doc.qt.io/qtforpython-6/

fpdf2. (2026). fpdf2 documentation. https://py-pdf.github.io/fpdf2/

Rojas, M. (2026). Clases de la Unidad 2: registros de longitud fija, CRUD, organización de archivos e índices. Universidad Rafael Landívar.